

Using optimization methods to reduce the computational complexity of mine planning problems

1. Block-Angular Structure

A matrix A has a **block angular structure** if it is of this form:

$$A = \begin{bmatrix} A_1 & A_2 & \cdots & A_{n-1} & A_n \\ A_{n+1} & 0 & \cdots & 0 & 0 \\ 0 & A_{n+2} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & A_{2n} \end{bmatrix}$$

where A_i are submatrices of size $m_i \times n_i$. We call the first n matrices $A_1, A_2, \dots, A_{n-1}$ the bounding constraints and the last n matrices the independent constraints. Notice that this easily becomes a linear program that looks like this,

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n \vec{c}_i \vec{x}_i \\ & \text{subject to} && [A_1, \dots, A_n, I] \vec{x}^T = \vec{b}_0 \quad \text{linking constraints} \\ & && A_{n+i} \vec{x}_i = \vec{b}_i, \quad \forall i \in \{1, \dots, n\} \quad \text{independent constraints} \\ & && \vec{x}^T \geq 0, \quad \vec{x}_i \geq 0, \quad \forall i \in \{1, \dots, n\} \quad \text{non-negativity constraints} \end{aligned}$$

In this linear program, the $\vec{c}_i$ are the costs and $\vec{x} = [\vec{x}_1, \dots, \vec{x}_n]^T$ are the decision variables and $\vec{b}_i$ are the right hand side of the constraints. The aim of the Dantzig-Wolfe decomposition is to be more computationally efficient in solving this problem. We start with a **restricted master problem** (RMP), which usually encompasses all the linking constraints and then check the independent constraints one by one by solving them as subproblems.

2. Column Generation

2.1 Intro

Column generation is a method for solving large scale linear programs. It is a variant of the simplex algorithm that is used to solve linear programs with a large number of variables. The idea is to start with a small subset of the variables and then iteratively add variables to the problem until the solution is optimal. We have a **master problem** (MP) and our **restricted master problem** (RMP). The RMP is obtained by taking a subset of the variables from the MP based on some heuristic. We iteratively solve the RMP and then add variables to the RMP based on positive reduced cost (in the context of max problems, negative reduced cost for min problems). This process is repeated until the MP is optimal, which is when we have no positive reduced cost columns.

2.2 Algorithm details

Given a Master Problem (MP), and its dual,

$$\begin{aligned} \max \quad & c^T x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Now, given our master problem, we define a restricted master problem (RMP) with a subset of the variables from the master problem, I , based on having a feasible solution and some heuristic (for finding 'important' variables).

$$\begin{aligned} \max \quad & \sum_{j \in I} c_j x_j \\ \text{s.t.} \quad & \sum_{j \in I} A_j x_j = b \\ & x_j \geq 0, \quad \forall j \in I \end{aligned}$$

After defining the RMP, we perform master iterations until we cannot find a positive reduced cost column. A master iteration consists of solving the RMP and then looking for a variable x_j with a positive reduced cost. We do this by solving the column generation subproblem (CGS), $\min_{i=1, \dots, n} \bar{c}_i$.

Now,

2.3 Column Generation for the Cutting Stock Problem

A common example of column generation is using it to solve the cutting stock problem more efficiently. In this problem, we have a set of items that we want to cut from a stock item. We want to minimize the waste, i.e. the amount of material that is left over after cutting the items. Our minimization problem looks like this,

$$\begin{aligned} \min & \sum_{j=1}^m w_j x_j \\ \text{s.t.} & \sum_{j=1}^m a_{ij} x_j = b_i \quad \forall i \in \{1, \dots, n\} \\ & x_j \geq 0 \quad \forall j \in \{1, \dots, m\} \end{aligned}$$

where w_j is the waste of the j th pattern, x_j is the number of times the j th pattern is used, a_{ij} is the number of times the i th item appears in the j th pattern, and b_i is the number necessary of the i th item based on the orders.

We now create a restricted master problem (RMP) by taking a subset of the patterns from the master problem, I . A common heuristic is to take the patterns that contain the max of each order as this initial set.

$$\begin{aligned} \min & \sum_{j \in I} w_j x_j \\ \text{s.t.} & \sum_{j \in I} a_{ij} x_j = b_i \quad \forall i \in \{1, \dots, n\} \\ & x_j \geq 0 \quad \forall j \in \{1, \dots, m\} \end{aligned}$$

Now, we need to solve the column generation subproblem (CGS) to find the column with the minimum reduced cost. This is a knapsack problem of the form,

$$\begin{aligned} \min_i \quad & \bar{c}_i = c_i - \sum_{j \in I} y_j a_{ij} \\ \text{s.t.} \quad & \sum_{j \in 1, \dots, m} a_{i,j} w_j \leq W \\ & a_{i,j} \geq 0 \quad \forall j \in \{1, \dots, m\} \end{aligned}$$

where w_j is the size of the order j and y_j is the dual variable for the order j . Now, we can rewrite this problem as simply,

$$\begin{aligned}
& \max_i \quad \sum_{j \in I} y_j a_{ij} \\
& \text{s.t.} \quad \sum_{j \in 1, \dots, m} a_{i,j} w_j \leq W \\
& \quad a_{i,j} \geq 0 \quad \forall j \in \{1, \dots, m\}
\end{aligned}$$

Now after we solve the CGS, if our value is ≥ 0 , then we know that the current solution to the RMP is optimal. If it is less than 0, then we know that we can improve the solution to the RMP by adding the column corresponding to the CGS solution. We then add this column to the RMP, solve again and repeat until we have no positive reduced cost columns.

2.3.2 Implementation

We implemented this in Python using the Gurobi Optimizer. We used the Gurobi Optimizer because it is a state of the art solver for linear programs and column generation. We used the Python interface for Gurobi and the column generation algorithm is implemented in the Gurobi Python API. Consider the following pseudocode for the column generation algorithm,

2.3.3 Simple Example

Consider the following example, we have $W = 15$, with three orders, $w_1 = 4, w_2 = 6, w_3 = 7$, for which we respectively have $b_1 = 80, b_2 = 50, b_3 = 100$.

We start by creating an RMP using the heuristic of taking the patterns that contain the max of each order. This means we have three columns in the RMP,

$$a_1 = \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix}, \quad a_2 = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}, \quad a_3 = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix}$$

So our RMP looks like this,

$$\begin{aligned}
\min \quad & x_1 + x_2 + x_3 \\
\text{s.t.} \quad & 3x_1 = 80 \\
& 2x_2 = 50 \\
& 2x_3 = 100 \\
& x_1, x_2, x_3 \geq 0
\end{aligned}$$

We now solve the RMP and find that the solution is $x_1 = 80/3, x_2 = 25, x_3 = 50$. We now need to solve the CGS to find the column with the minimum reduced cost. Our CGS is the following, $\min_i 1 - \sum_{j \in P} y_j a_{ij}$, which we can rewrite as a maximization problem,

$$\begin{aligned}
\max \quad & a_1 y_1 + a_2 y_2 + a_3 y_3 = \frac{1}{3}a_1 + \frac{1}{2}a_2 + \frac{1}{2}a_3 \\
\text{s.t.} \quad & 4a_1 + 6a_2 + 7a_3 \leq 15 \\
& a_1, a_2, a_3 \geq 0
\end{aligned}$$

We solve this and find that the solution is $a_1 = 2, a_2 = 0, a_3 = 1$. We now check the reduced cost of this column,

$$1 - \frac{1}{3} \cdot 2 - \frac{1}{2} \cdot 0 - \frac{1}{2} \cdot 1 = -\frac{1}{6}$$

Since this is negative, we add this column to the RMP and solve again. We solve the RMP again and get an improved solution. Now we solve the CGS again, however this new solution, $a_1 = 2, a_2 = 1, a_3 = 0$ has a positive reduced cost which implies our solution is optimal and so we terminate.

3. Dantzig-Wolfe Decomposition

3.1 Overview

We'll start by giving a high level view of the algorithm, which is basically doing these steps (Wikipedia):

1. Start with a feasible solution to the RMP, formulate new objective functions for each subproblem such that the subproblems will offer solutions that improve the current objective of the master program.
2. Re-solve the subproblems given their new objective functions, the optimal value for each subproblem is offered to the master program.
3. Add one or all new columns generated by the solutions to the subproblems to the RMP based on respective ability to improve the objective.
4. Perform x iterations of the simplex algorithm in the master program, where x is the number of columns incorporated.
5. If the objective is improved, loop. Otherwise, return the solution.

D-W Decomposition is a special case of column generation where we have a block angular structure for our constraints. Our master problem looks like this,

$$\begin{aligned} \max \quad & c^T x \\ \text{s.t.} \quad & \begin{bmatrix} A_1 & A_2 & \cdots & A_{n-1} & A_n \\ A_{n+1} & 0 & \cdots & 0 & 0 \\ 0 & A_{n+2} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & A_{2n} \end{bmatrix} x = b \\ & x \geq 0 \end{aligned}$$

3.2 Algorithm Details

Now, since all the blocks in the independent constraint blocks are independent, we can restate the master problem as,

$$\begin{aligned} \max \quad & c^T x \\ \text{s.t.} \quad & [A_1, \dots, A_n]x = b_0 \\ & x_i \in S_i \quad \forall i \in \{1, \dots, n\} \end{aligned}$$

We denote the feasible region of the subproblems as

$$S_i = \{x_i \geq 0 | A_{n+i}x_i = b_i\}$$

Assume that S_i is bounded for simplicity (it can easily be extended to having unbounded regions, we would consider unbounded rays). We have a set of vertices $V_i = \{(v_{i,1}, \dots, v_{i,n}) | v_{i,j} \in S_i\}$ which completely defines the feasible region of the subproblem. Since the feasible region is convex, we can represent every $x_i \in S_i$ as a convex combination of the vertices $v_{i,j}$, i.e. $x_i = \sum_{j=1}^n \lambda_{i,j} v_{i,j}$ and with $\sum_{j=1}^n \lambda_{i,j} = 1$.

Now, we can transform the RMP by using the $\lambda_{i,j}$ as the new decision variables.

$$\begin{aligned} \max \quad & \sum_{i=1}^n \sum_{j=1}^{N_i} \lambda_{i,j} (c_i v_{i,j}) \\ \text{s.t.} \quad & \sum_{i=1}^n \sum_{j=1}^{N_i} \lambda_{i,j} (A_i v_{i,j}) = b_0 \\ & \sum_{j=1}^{N_i} \lambda_{i,j} = 1 \quad \forall i = 1, \dots, n \\ & \lambda_{i,j} \geq 0 \quad \forall i, j \end{aligned}$$

Notably, we reduced the number of constraints to $n + n_0$ (the number of binding constraints). However, there are now substantially more variables, which is why we will use an altered simplex method. We now proceed to create an RMP with a subset of the lambdas. It is important to note that we need to include necessary constraints to ensure the solution is feasible.

After creating and solving the RMP, we proceed by creating dual variables, $y_0 \in \mathbb{R}^{n_0}$ for the binding constraints and $z \in \mathbb{R}^n$ for the lambdas. Note that each column represents one vertex. And so, we want to find out which columns have positive reduced cost (because this is a maximization problem). If we consider the column for $\lambda_{i,j}$, then the reduced cost is

$$\bar{c}_i = c_i v_{i,j} - y_0 A_i v_{i,j} - z_i$$

So similar to the subproblem in the column generation algorithm, we will solve the following CGS,

$$\begin{aligned} \min_i \quad & y_0 A_i v_{i,j} + z_i \text{ s.t.} \quad A_i v_{i,j} = b_i \\ & v_{i,j} \geq 0 \end{aligned}$$

Now, similar to the column generation algorithm, we solve the CGS for each subproblem and check if the reduced cost is positive (since this is a maximization problem). If it is, we add the column to the RMP and solve again.

If it is not, we know that the current solution to the RMP is optimal and we terminate.

The main difference between the D-W Decomposition and the Column Generation algorithm is the step where we consider the subproblems w.r.t. the lambdas of the subproblems.

3.3 Example

Consider the following example,

4. Block Selection Linear Program

4.1 Defining the problem

This is a subproblem of larger mine planning optimization problems. The objective is to maximize the total value of the blocks selected, subject to the constraints that the total volume of the selected blocks does not exceed the available volume, and that the total value of the selected blocks does not exceed the available value. Here is a formal version of the problem for two modes, $\{A, B\}$ and two ore types $\{I, II\}$.

$$\begin{aligned}
& \max \sum_{b \in \mathcal{B}} (V_{b,A,I} m_{b,A,I} + V_{b,A,II} m_{b,A,II} + V_{b,B,I} m_{b,B,I} + V_{b,B,II} m_{b,B,II}) \\
& \text{st.} \quad \sum_{b \in \mathcal{B}} \left(\frac{m_{b,A,I}}{r_{A,I}} + \frac{m_{b,A,II}}{r_{A,II}} + \frac{m_{b,B,I}}{r_{B,I}} + \frac{m_{b,B,II}}{r_{B,II}} \right) \leq t_{\text{period}} \quad (1) \\
& r_{A,I} m_{b,A,I} + r_{A,II} m_{b,A,II} = 0 \quad \forall b \in \mathcal{B} \quad (2) \\
& r_{B,I} m_{b,B,I} + r_{B,II} m_{b,B,II} = 0 \quad \forall b \in \mathcal{B} \quad (3) \\
& m_{b,A,I} + m_{b,B,I} \leq m_{b,I} \\
& m_{b,A,II} + m_{b,B,II} \leq m_{b,II} \quad \forall b \in \mathcal{B} \quad (4) \\
& m_{ijk}, r_{jk} \geq 0 \quad \forall i \in \mathcal{B}, j \in \text{modes}, k \in \text{ores}
\end{aligned}$$

where $\mathcal{B}$ is the set of blocks, V_{bjk} is the value of block b in mode j and ore k , m_{bjk} is the volume of block b in mode j and ore k , r_{jk} is the recovery rate of ore k in mode j , and t_{period} is the total time available for block selection.

The first constraint represents the time constraint, that is to say that the total volume of blocks divided by the rate of recovery of the ore must be less than or equal to the total time available. The second and third constraints represent the volume constraints, that is to say that the volume of blocks selected in mode j and ore k must be less than or equal to the total volume of blocks in mode j and ore k .

4.2 Subproblems

This linear program is an ideal application of Dantzig-Wolfe because of the block angular structure of the constraints, but also because the subproblems

are quite simple and all the same. Basically our feasible region will always look like this,

4.2.1 Column Generation

4.2.2 Dantzig-Wolfe

4.3 Implementation details

4.4 Computational Analysis

4.5 Example with 2 modes and 2 ores

References

- [1] Hillier, F. S., & Lieberman, G. J. (2015). Introduction to Operations Research (10th ed.). McGraw-Hill Education.
- [2] Cornell University. “Column Generation Algorithms.” Optimization @ Cornell CBE. https://optimization.cbe.cornell.edu/index.php?title=Column_generation_algorithms
- [3] Cornell University. “Dantzig-Wolfe Decomposition.” Optimization @ Cornell CBE. https://optimization.cbe.cornell.edu/index.php?title=Dantzig-Wolfe_decomposition
- [4] Wikipedia. “Column Generation.” Wikipedia. https://en.wikipedia.org/wiki/Column_generation
- [5] Wikipedia. “Dantzig-Wolfe decomposition.” Wikipedia. https://en.wikipedia.org/wiki/Dantzig-Wolfe_decomposition
- [6] Poirrier, E. (n.d.). Column Generation. Retrieved from <https://www.poirrier.ca/courses/co353/d/colgen.pdf>